DATA 200: Python Learning Log

Fictional Sample Entries: Week 1 Completed Example

Instructor note. These entries are fictional models showing the expected level of detail. Students may document relevant homework, guided examples, or class exercises when those activities genuinely contributed to their Python learning. Students must write their own narratives, submit their own code, describe their own debugging process, and provide screenshots from their own successful program runs. Each entry below marks the place for that screenshot with a placeholder.

1 Entry 1: Temperature Conversion

Time spent: 50 minutes

Learning focus: Variables, user input, numeric conversion, and basic arithmetic

Resources used: Class notes, the introductory Python video, and completed in-class practice problems

Python file: `week01_entry01_temperature_converter.py`

Progress Narrative

Today I worked on learning how Python stores values in variables and how user input can be incorporated into a program. I reviewed my notes from class and reworked one of the temperature-conversion examples discussed during class. My initial version produced an error because I forgot that values entered through `input()` are stored as text. After reviewing the example, I converted the input to a numeric data type using `float()`. Once I made that change, the calculation worked correctly. I tested several values, including a negative temperature, to determine whether the program produced reasonable results. By the end of this practice session, I felt more comfortable using variables, performing calculations, and displaying formatted output.

Program Purpose

This program asks the user to enter a temperature in degrees Celsius and converts it to degrees Fahrenheit. It demonstrates user input, conversion from text to a floating-point number, variable assignment, arithmetic operations, and formatted output.

Listing 1: Temperature-conversion program.

```
"""Convert a temperature from degrees Celsius to degrees Fahrenheit.

Fictional sample log entry: a model of the expected level of detail,
not real student work.
"""

```

```python
# Ask the user for a temperature and convert the typed text to a decimal number.
celsius = float(input("Enter temperature in Celsius: "))

# Apply the Celsius-to-Fahrenheit conversion formula.
fahrenheit = (celsius * 9 / 5) + 32

# Display the converted value with two digits after the decimal point.
print(f"Temperature in Fahrenheit: {fahrenheit:.2f}")
```

Run Result

The program successfully converted several test values. Entering `0` produced `32.00` degrees Fahrenheit, and entering `100` produced `212.00` degrees Fahrenheit. These results matched the values I calculated separately.



Figure 1: Successful execution of the Entry 1 program. In your own log, this is a screenshot of your program running on your own computer.

2 Entry 2: Grade Classification

Time spent: 65 minutes

Learning focus: Conditional statements and decision-making

Resources used: A homework problem, class examples, and Python reference materials

Python file: `week01_entry02_grade_classifier.py`

Progress Narrative

Today I practiced using `if`, `elif`, and `else` statements to make decisions within a program. I adapted a homework-style problem in which a numeric score is converted into a letter grade. My first attempt incorrectly classified some boundary values because I used greater-than operators when I needed greater-than-or-equal-to operators. After testing scores such as 90, 80, and 70, I corrected the conditions and obtained the expected output. This exercise helped me understand how program flow changes according to a condition. I also learned

that boundary values are important test cases because a program can appear to work while still treating exact cutoff values incorrectly.

Program Purpose

This program asks the user for a numeric score and displays the corresponding letter grade. It demonstrates ordered conditional statements, comparison operators, variable assignment, and formatted output.

Listing 2: Letter-grade classification program.

```
"""Classify a numeric score using a standard letter-grade scale.

Fictional sample log entry: a model of the expected level of detail,
not real student work.
"""

# Ask the user for a score and convert the response to a decimal number.
score = float(input("Enter your score: "))

# Select the letter grade. Checking from highest to lowest prevents overlap.
if score >= 90:
    letter_grade = "A"
elif score >= 80:
    letter_grade = "B"
elif score >= 70:
    letter_grade = "C"
elif score >= 60:
    letter_grade = "D"
else:
    letter_grade = "F"

print(f"Grade: {letter_grade}")
```

Run Result

I tested scores of `95`, `87`, `72`, and `55`; the program returned A, B, C, and F, respectively. I also tested exact boundary values such as `90` and `80` to confirm that the revised comparison operators worked correctly.



Figure 2: Successful execution of the Entry 2 program. In your own log, this is a screenshot of your program running on your own computer.

3 Entry 3: Average Calculator

Time spent: 75 minutes

Learning focus: Loops, accumulation, and calculating an arithmetic mean

Resources used: An in-class practice worksheet and additional homework exercises

Python file: `week01_entry03_average_calculator.py`

Progress Narrative

Today's practice focused on repetition using a `for` loop. During class, we worked through examples that repeated a calculation without requiring the same statements to be written several times. I created a program that collects five numeric values and computes their average. In my first attempt, I placed the running-total assignment inside the loop. That reset the total during every repetition and caused the final answer to be incorrect. I moved the total variable before the loop, reran the program, and obtained the expected result. This activity showed me how a loop can simplify repetitive work and how the location of a variable assignment can affect a calculation.

Program Purpose

This program requests five numbers from the user, adds them to a running total, and calculates their arithmetic mean. It demonstrates a `for` loop, `range()`, accumulation, numeric input, and formatted output.

Listing 3: Five-value average calculator.

```
"""Collect five numbers and calculate their arithmetic mean.

Fictional sample log entry: a model of the expected level of detail,
not real student work.
"""

```

```python
# The running total must be created before the loop so it is not reset.
total = 0.0
number_of_values = 5

for entry_number in range(1, number_of_values + 1):
    number = float(input(f"Enter number {entry_number}: "))
    total += number

average = total / number_of_values
print(f"Average = {average:.2f}")
```

Run Result

I entered `10`, `20`, `30`, `40`, and `50`. The program reported an average of `30.00`, which matched my independent calculation. I then tested a second set containing decimal values to confirm that using `float()` allowed non-integer input.



Figure 3: Successful execution of the Entry 3 program. In your own log, this is a screenshot of your program running on your own computer.

4 Entry 4: Data Summary

Time spent: 90 minutes

Learning focus: Lists and introductory descriptive statistics

Resources used: A homework assignment and class discussion about summarizing data

Python file: `week01_entry04_data_summary.py`

Progress Narrative

For this session, I worked on storing observations in a list and performing a simple data summary. In class, we discussed how an initial examination of a dataset often includes measures such as the minimum, maximum, and mean. I created a program that stores a small collection of values and calculates those three quantities. I first practiced accessing the list

and checking its length, and then used the built-in `sum()`, `len()`, `min()`, and `max()` functions. I also added comments that explain the purpose of each part of the program. Compared with my earlier code, this file is more organized and gives a reader more information about both the data and the calculations.

Program Purpose

This program stores a small dataset in a Python list and reports the original observations, minimum, maximum, and arithmetic mean. It demonstrates lists, built-in functions, arithmetic, comments, and formatted output.

Listing 4: Basic data-summary program.

```
"""Calculate basic summary statistics for a small example dataset.

Fictional sample log entry: a model of the expected level of detail,
not real student work.
"""

# Store the observations in a Python list.
data = [12, 18, 25, 30, 22, 16, 19]

# Calculate the mean by dividing the sum by the number of observations.
average = sum(data) / len(data)

# Display the original values and three basic summary statistics.
print("Data:", data)
print("Minimum:", min(data))
print("Maximum:", max(data))
print(f"Average: {average:.2f}")
```

Run Result

The program reported a minimum of `12`, a maximum of `30`, and an average of `20.29`. I checked the sum and number of observations separately and obtained the same rounded average.



Figure 4: Successful execution of the Entry 4 program. In your own log, this is a screenshot of your program running on your own computer.

5 Week 1 Reflection

Across these four sessions, I progressed from working with individual values to making decisions, repeating operations, and summarizing a collection of data. The most useful debugging lesson was that a program should be tested with carefully selected values, especially exact boundaries and simple cases for which the result can be calculated by hand. I also became more intentional about filenames and comments. My next goal is to add input validation so that programs respond appropriately when a user enters text or a value outside an expected range.